

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD

PIPELINE DESIGN EXERCISE

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO3 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	Ragul SG	Reg. No.:	192525007
Date:		Duration:	60 minutes

Code of Conduct

I RAGUL SG – 192525007 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the Student _____

TABLE OF CONTENTS

- 1. INTRODUCTION AND PROBLEM OVERVIEW**
- 2. CI/CD NEEDS AND PROJECT REQUIREMENTS**
- 3. PIPELINE WORKFLOW AND STAGES**
- 4. TOOLS AND TECHNOLOGIES**
- 5. AUTOMATED TESTING STRATEGY**
- 6. DEPLOYMENT STRATEGY AND ENVIRONMENTS**
- 7. SECURITY, QUALITY, NOTIFICATIONS AND ROLLBACK**
- 8. PIPELINE DIAGRAM AND END-TO-END WALKTHROUGH**
- 9. CONCLUSION**
- 10. APPENDIX**

1. INTRODUCTION AND PROBLEM OVERVIEW

1.1 PROBLEM STATEMENT

The assigned problem statement requires the design of a CI/CD (Continuous Integration and Continuous Deployment) pipeline for the FoodieConnect Online Food Ordering System — a three-tier web application in which registered customers browse restaurant menus, add items to a cart, place orders, make secure online payments, and track deliveries, while restaurant administrators manage the menu, accept or reject orders, and view sales reports. Because orders, payments, and refund workflows are business-critical, the system must be released frequently but safely: every change must be built, tested, and quality-checked automatically, and deployment to production must support instant rollback so that a customer's payment flow is never disrupted by a bad release. The assignment therefore asks for a pipeline design that automates code integration, building, testing, and deployment, with suitable tools, a testing strategy, a multi-environment deployment process, and built-in security, notification, and rollback mechanisms.

1.2 SYSTEM UNDER CONSIDERATION

The system is implemented as a Node.js application (Express API plus a React front-end) with a MySQL database and Razorpay as the payment gateway, deployed on AWS EC2 behind Nginx. The repository is hosted on GitHub and follows a feature-branch workflow with pull requests. This technology choice directly shapes the pipeline: the build stage must run npm install and npm run build, the test stage runs Jest unit tests and Playwright end-to-end tests, and deployments target Docker containers on EC2. The high-level architecture of the system is reproduced below for reference.

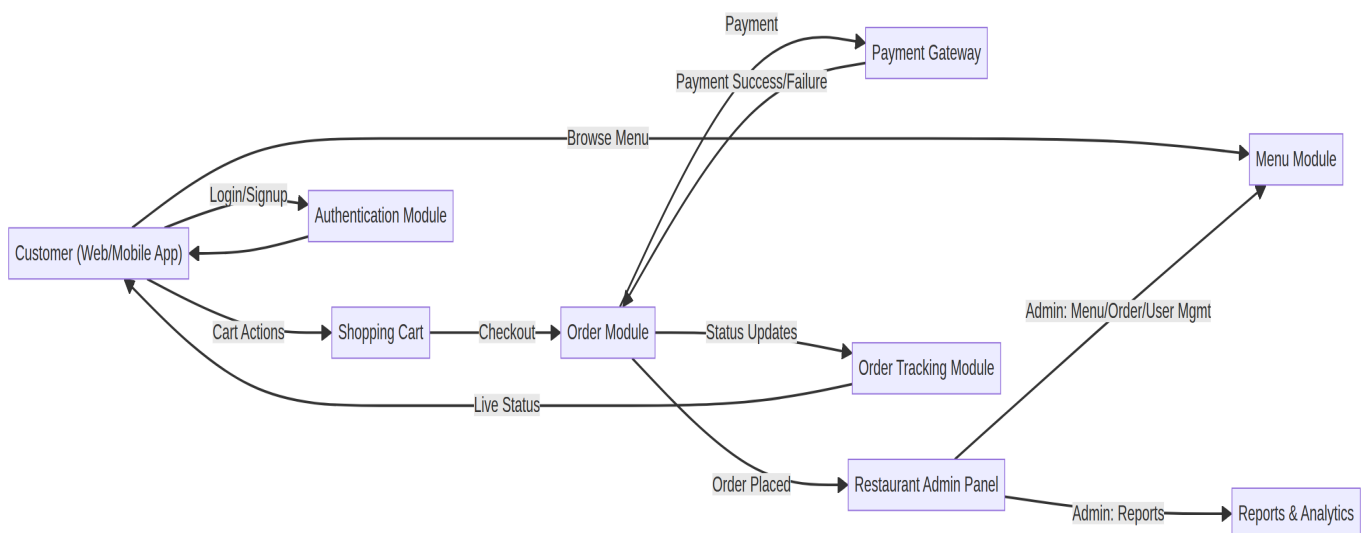


Figure 1: High-level architecture of the FoodieConnect system (the subject of this pipeline design).

2. CI/CD NEEDS AND PROJECT REQUIREMENTS

The CI/CD needs of the assigned problem statement were derived from the system's operational profile. FoodieConnect serves time-sensitive transactions (order placement, payment capture, refund processing), so the pipeline must guarantee that only code that passes an automated gate can reach customers, and that any production regression can be undone in minutes rather than hours. The identified needs and the pipeline feature that satisfies each one are summarised below.

Fast integration feedback	Developers must know within minutes whether a change compiles and passes unit tests	Automatic build and test on every commit to a feature branch
Payment-critical test gating	A broken checkout or refund flow would directly lose revenue	Blocking quality gates: coverage $\geq 80\%$, SonarQube quality gate, npm audit clean
Safe staged releases	New features (offers, menu changes) should be validated on realistic data before customers see them	Three environments: DEV $\rightarrow$ STAGING $\rightarrow$ PRODUCTION with promotion gates
Zero-downtime deployment	Ongoing orders and payments must not be interrupted mid-release	Blue/green deployment behind Nginx with health-check cutover
Instant rollback	A payment gateway integration defect must be reverted immediately	Previous green image retained and switchable in under 60 seconds
Visibility and alerts	The team must be notified of every build outcome and every failed deployment	Slack notifications on each stage; PagerDuty for production health alarms
Security assurance	Customer credentials and payment data must never ship with a known vulnerability	TruffleHog secret scan, SAST with CodeQL, dependency audit, and TLS checks
Auditability	Every released version must be traceable to a commit, a build number, and an approver	Git tags on releases; immutable Docker images tagged with the commit SHA

3. PIPELINE WORKFLOW AND STAGES

3.1 STAGE OVERVIEW

The pipeline is organised into seven stages that map one-to-one onto the workflow requested by the assignment: source code management, build, testing, code quality analysis, packaging, deployment, and post-deployment verification. Each stage has a defined entry condition, a set of automated checks, and an exit gate; a stage cannot complete unless its gate is satisfied, which is what makes the pipeline a trustworthy delivery mechanism rather than a set of loose scripts.

1. Source code management	Commit or pull request on GitHub	Feature branch workflow; PR reviews; branch-protection rules on main	Merged PR on main
2. Build	Push/merge to main	npm install with dependency audit, npm run build for API and UI	Build artefact produced with zero errors
3. Testing	Build success	Jest unit tests, Jest integration tests against a seeded test database, Playwright E2E tests	100% of tests pass
4. Code quality analysis	Test success	ESLint linting, SonarQube static analysis, coverage report	Zero critical issues; coverage $\geq 80\%$
5. Packaging	Quality gate passed	Docker image built and tagged with commit SHA; pushed to GitHub Container Registry	Image published to GHCR
6. Deployment	Image available + approval	Auto-deploy to DEV; manual promotion to STAGING; gated release to PRODUCTION	Health checks pass on target
7. Post-deployment verification	Deployment complete	Smoke tests, monitoring, error-rate watch, auto-rollback decision	Release marked complete or rolled back

3.2 BRANCHING MODEL AND PIPELINE TRIGGERS

The repository uses a feature-branch workflow with a protected main branch. Every change starts as a pull request; merging to main triggers the CI workflow (lint, security scan, unit tests, coverage, and build), and the continuous deployment portion is triggered only by tags: tags of the form v1.x-staging deploy to the staging environment, while a v1.x-release tag — which can only be created after the staging approval gate — deploys to production. This separation guarantees that no code ever reaches customers without having passed staging acceptance.

Pull request to main	CI: lint + security scan + unit tests + build	Feedback to PR; no deployment
Merge to main	CI workflow + auto-deploy	DEV environment
Tag v1.x-staging	Full pipeline: CI + image build + staging deploy	STAGING environment
Tag v1.x-release (after approval)	Production deploy with blue/green cutover	PRODUCTION environment

4. TOOLS AND TECHNOLOGIES

Each stage of the pipeline is supported by a specific tool. The selection was driven by three criteria: fit with the project's technology stack (JavaScript/Node.js on AWS EC2), zero or low cost for a small team, and tight GitHub integration so that triggers, approvals, and artefacts remain in one place. The complete tool map and the justification for each choice are given below.

SCM	GitHub + feature branches	Source of truth, PR reviews, protected main	Required by the assignment; free, ubiquitous, and integrates natively with Actions
Orchestration	GitHub Actions	Runs the entire CI/CD workflow as YAML	YAML-defined workflows live in the repository itself; 2,000 free minutes/month; no extra server to maintain

Build	npm + Docker	Dependency install, build, container image	npm is the native package manager; Docker guarantees identical artefacts from DEV to PRODUCTION
Registry	GitHub Container Registry (GHCR)	Stores versioned Docker images	Co-located with the repository; immutable image tags by commit SHA enable reliable rollbacks
Testing	Jest, Playwright	Unit/integration tests and E2E browser tests	Jest is the Node.js default with coverage built in; Playwright tests the real checkout UI the way customers use it
Quality	ESLint, SonarQube, Coveralls	Linting, static analysis, coverage reporting	Catches code smells and security hotspots; coverage gate enforces the 80% threshold
Security	TruffleHog, CodeQL, npm audit	Secret scan, SAST, dependency audit	Three complementary scans: leaked credentials, source-code vulnerabilities, and vulnerable third-party packages
Deploy	SSH/Ansible to AWS EC2	Pull image and roll containers	EC2 is the production host; agentless SSH deploy is simple and auditable for a two-person team
Proxy	Nginx	Blue/green traffic switching	Free, battle-tested; atomic upstream switch means zero-downtime cutover
Monitoring	Prometheus, Grafana	Collects and visualises deployment metrics	Open-source; Grafana alerting feeds the rollback decision
Notifications	Slack, PagerDuty	Build status chat alerts; production incidents	Instant team visibility; PagerDuty escalates unacknowledged production alarms

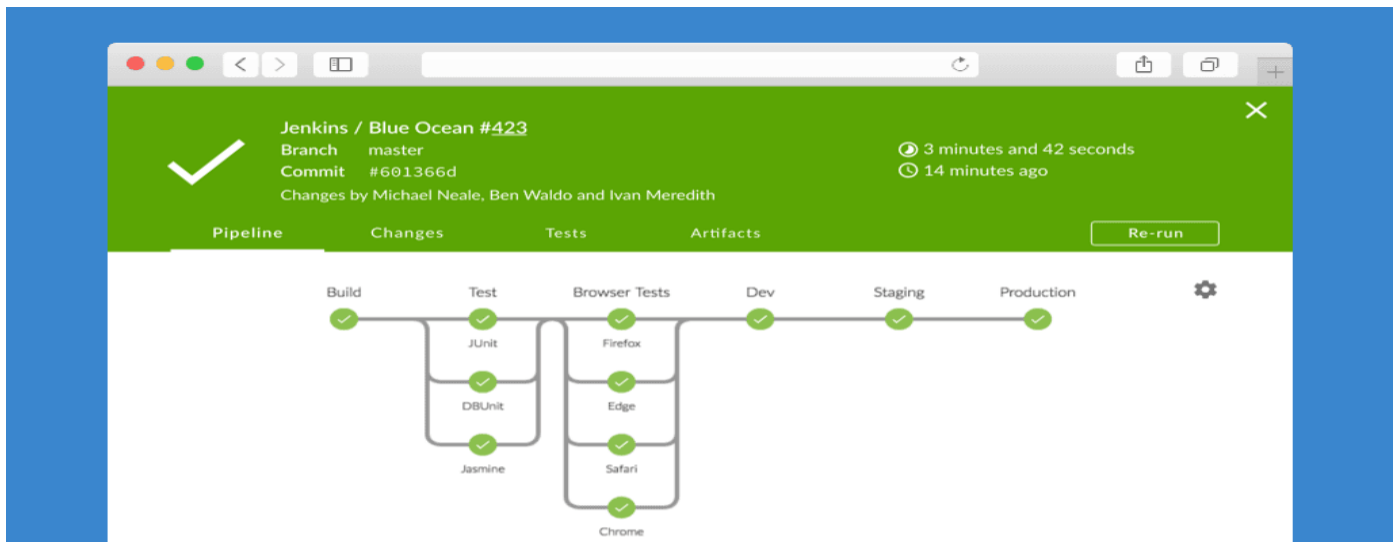


Figure 2: Typical CI/CD pipeline visualisation as rendered by Blue Ocean (illustrative of the stage progress shown by this design).

5. AUTOMATED TESTING STRATEGY

The testing strategy is structured as a pyramid: many fast unit tests run on every commit, a smaller set of integration tests runs against a seeded test database, and a focused set of end-to-end tests runs only at staging, where the full stack including the mock payment gateway is available. Tests are treated as pipeline gates — a failure at any level stops the build and blocks deployment.

Unit tests	Cart totals, delivery-fee slabs, order-state transitions, discount logic	Jest	Every push and PR	100% pass; line coverage $\geq 80\%$
Integration tests	API endpoints against a seeded MySQL test instance (Docker)	Jest + Supertest	Every push and PR	100% pass
Security tests	npm audit, CodeQL SAST rules, TruffleHog secret scan	npm audit, CodeQL, TruffleHog	Every PR and merge	Zero critical/high findings
E2E tests	Full user journey: login, add-to-cart, checkout, order tracking	Playwright (headless Chromium)	On staging deploy	100% pass

Smoke tests	Health endpoint, menu API, login page availability	Custom curl-based script	Immediately after every deployment	HTTP 200 within 30 s
Performance smoke	Home page load <3 s, checkout <2 s under staging load	k6	Weekly + before production release	All thresholds met

The delivery-fee and minimum-order business rules are verified with parameterised Jest tests covering the boundary values of each rule (Rs.98 vs Rs.99 minimum order; 5.0 km vs 5.1 km fee slabs), so that the same rules checked in the earlier test-case design exercise are re-verified on every commit. The E2E suite additionally covers the payment-failure path by forcing the mock gateway to decline, confirming that the cart is preserved and the retry flow works — the highest-value negative scenario in the entire system.

6. DEPLOYMENT STRATEGY AND ENVIRONMENTS

6.1 ENVIRONMENT LAYOUT

Three environments are maintained, each mirroring the next in configuration so that a promotion carries no surprises.

The diagram below summarises the layout and the promotion path.

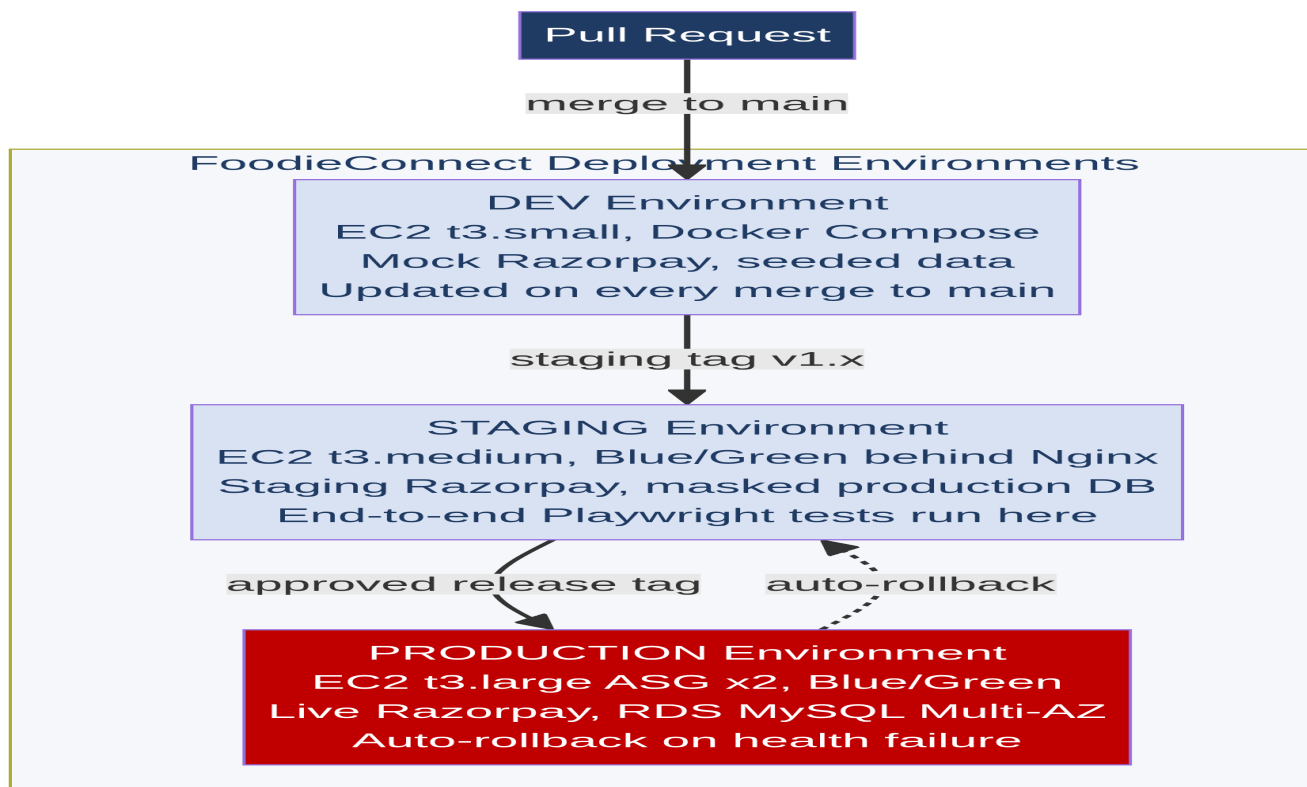


Figure 3: Deployment environment layout and promotion path.

DEV	EC2 t3.small, Docker Compose	Seeded synthetic data; mock Razorpay	Automated on merge to main	Developer and QA validation of new features
STAGING	EC2 t3.medium, blue/green behind Nginx	Masked production database snapshot; staging Razorpay	Manual promotion after CI (v1.x-staging tag)	E2E, UAT, and pre-release acceptance
PRODUCTION	EC2 t3.large in an Auto Scaling group (2 instances), blue/green	Live Razorpay; RDS MySQL Multi-AZ	Gated release (v1.x-release tag + approver)	Serving real customers

6.2 BLUE/GREEN DEPLOYMENT PROCESS

Production releases use a blue/green strategy. At any moment, Nginx routes customer traffic to the blue instance running the current version while the green instance is provisioned with the new Docker image. The pipeline then executes smoke tests against the green instance; only when the health endpoint, the menu API, and the login page all respond with HTTP 200 does Nginx atomically switch traffic to green. The blue instance is retained for the next release and as the immediate rollback target, which means a failed release is undone by re-pointing Nginx at blue — a single command that takes under a minute and requires no rebuild.

6.3 PROMOTION GATES

Promotion between environments is controlled by explicit gates. The staging promotion requires the CI workflow to be green and a repository owner approval on the workflow run; the production promotion additionally requires the E2E suite to be green on staging and a named approver (the team lead) to approve the release workflow. GitHub environments with required reviewers enforce these gates at the platform level, so no tag-based deployment can bypass them.

7. SECURITY, QUALITY, NOTIFICATIONS AND ROLLBACK

The assignment asks for security checks, quality checks, notifications, and rollback mechanisms to be incorporated directly into the pipeline. Their placement and behaviour are detailed below.

7.1 SECURITY CHECKS

Credential leak scan	TruffleHog	On every PR and merge	Pipeline fails; PR cannot merge
Static application security testing	CodeQL (GitHub-native rules)	On every PR	Fails on critical/high security advisories
Dependency vulnerability audit	npm audit + Dependabot alerts	On every PR and weekly schedule	Fails on critical/high severity
Transport security	TLS certificate verification against staging	On staging smoke tests	Alerts team; blocks production tag if missing

7.2 QUALITY CHECKS

Code style and formatting	ESLint + Prettier	Zero lint errors	Blocking gate in CI
Static analysis and code smells	SonarQube	No new critical issues; maintainability rating A/B	Blocking quality gate
Test coverage	Jest + Coveralls	Minimum 80% line coverage	Blocking gate; failing PRs cannot merge
Test pass rate	Jest + Playwright	100%	Blocking gate

7.3 NOTIFICATIONS

A single Slack channel (#foodieconnect-ci) receives structured messages at four points in the pipeline: on pipeline start (commit, author, branch), on CI failure (stage, failing test, log link), on staging deployment (version, changelog link, approval reminder), and on production release (version, deployer, post-deploy smoke result). Production health alarms beyond Slack are routed to PagerDuty so that a failed deployment at 3 a.m. is escalated to an on-call engineer within five minutes.

7.4 ROLLBACK MECHANISMS

Rollback operates at three levels. At the image level, every release keeps the previous Docker image in GHCR and on the EC2 instance, so reverting is a container restart with the old tag. At the traffic level, the blue/green Nginx switch reverts customer traffic in seconds without touching the new image at all. At the database level, schema migrations are written to be backward-compatible, and RDS snapshots taken before each production release allow point-in-time restore within the 24-hour retention window. The pipeline also performs automatic rollback: if the error rate on green exceeds 2% or the health endpoint fails within ten minutes of cutover, a Grafana alert triggers the revert workflow that switches Nginx back to blue and notifies the team.

8. PIPELINE DIAGRAM AND END-TO-END WALKTHROUGH

8.1 THE CI/CD PIPELINE DIAGRAM

The complete pipeline — from the developer's first commit to the released version monitored in production — is shown below. This is the principal artefact of the submission.

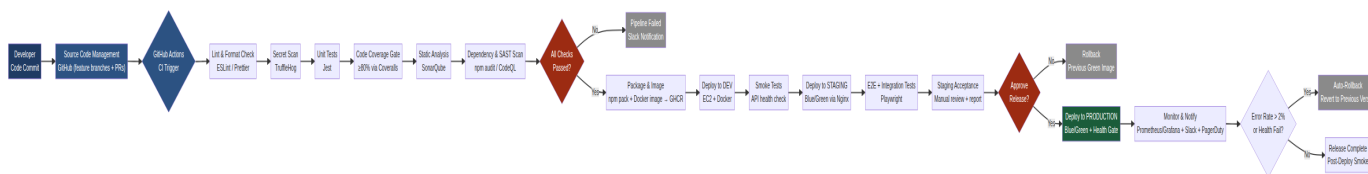


Figure 4: The FoodieConnect CI/CD pipeline from code commit to production deployment, including rollback paths.

8.2 WORKFLOW WALKTHROUGH: FROM COMMIT TO DEPLOYMENT

The workflow begins when a developer commits a change to a feature branch and opens a pull request. GitHub Actions immediately runs the CI workflow: ESLint checks the code style, TruffleHog scans for accidentally committed secrets, Jest unit tests verify the business logic, Coveralls confirms that coverage remains at or above 80%, and the application is built. Any failure is posted to Slack and the PR cannot be merged until it is fixed.

Once two reviewers approve the PR, it is merged into main. The merge re-runs CI and then automatically deploys the built image to the DEV environment, where developers and QA verify the feature against seeded data. When the feature is judged ready, a maintainer applies a v1.x-staging tag; the pipeline packages the application as a Docker image tagged with the commit SHA, publishes it to GHCR, deploys it to staging, and runs the Playwright end-to-end suite against the staging stack. The staging acceptance result is reviewed, and a named approver triggers the release.

For production, a v1.x-release tag is created and the blue/green deployment proceeds: the new image starts on the green instance, smoke tests and a ten-minute error-rate watch follow, and Nginx cutover completes the release. If the watch detects an error rate above 2% or a failing health endpoint, the auto-rollback workflow switches traffic back to blue and raises a PagerDuty incident, so the customer-facing impact of a bad release is measured in minutes, not hours. The release is finally announced on Slack with the version, the deployer, and the smoke-test outcome, giving the entire team a single auditable record of every deployment.

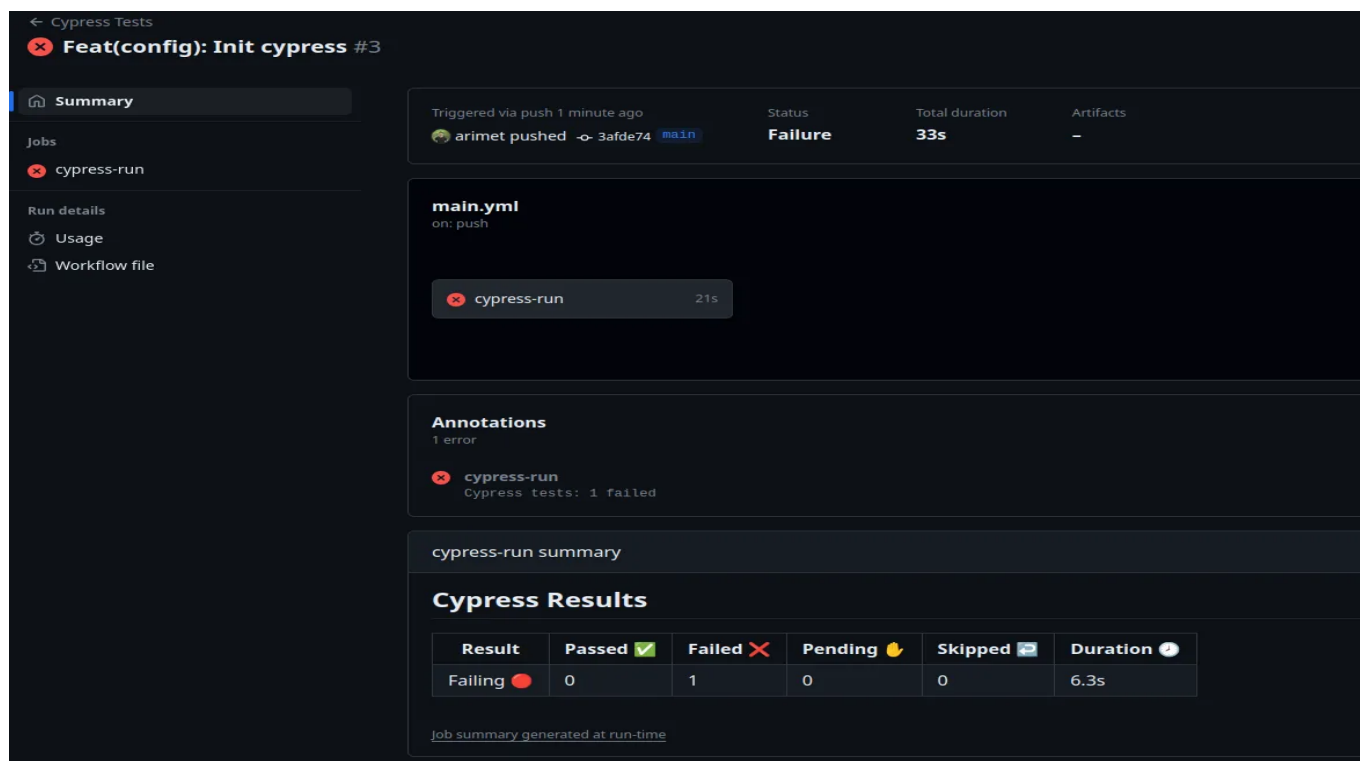


Figure 5: A GitHub Actions run reporting E2E results, as the pipeline would annotate each run (illustrative).

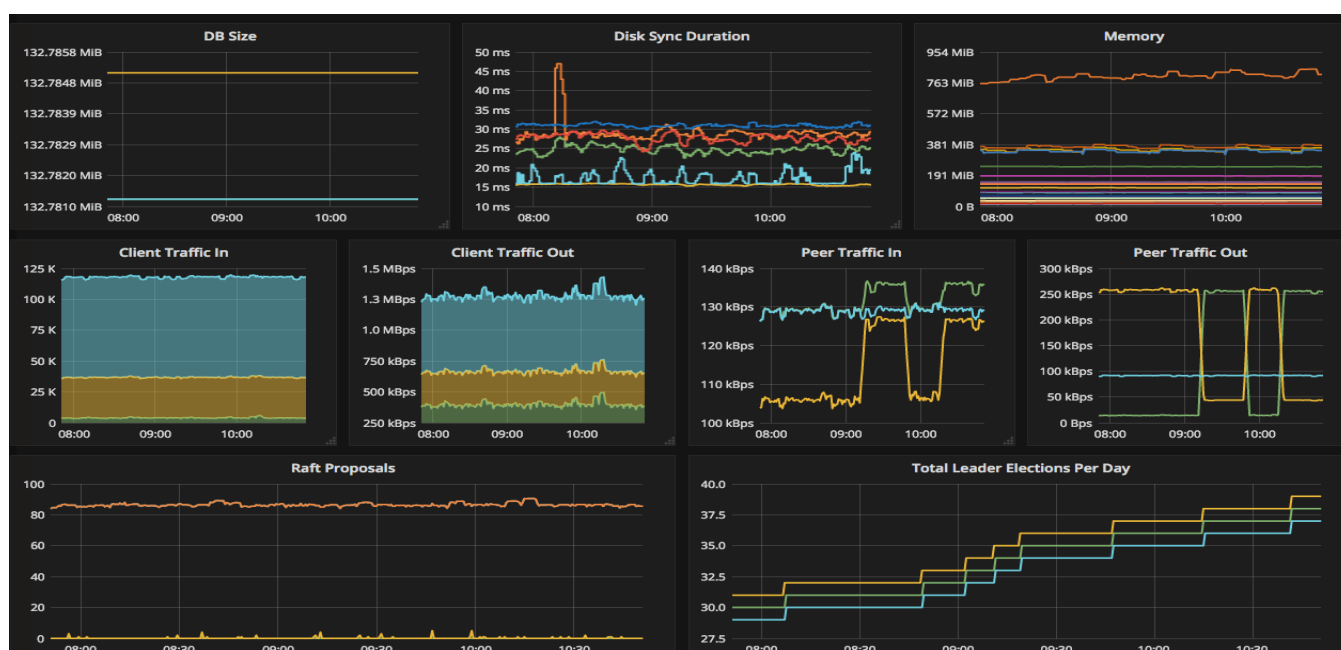


Figure 6: Monitoring dashboard supporting the post-deployment error-rate watch and rollback decision (illustrative).

9. CONCLUSION

This report has designed a complete CI/CD pipeline for the FoodieConnect Online Food Ordering System as required by the assigned problem statement. The pipeline's CI/CD needs were derived from the system's business-critical order and payment workflows; the seven pipeline stages (SCM, build, testing, quality analysis, packaging, deployment, and post-deployment verification) were each assigned appropriate tools — GitHub Actions, Jest, Playwright, SonarQube, CodeQL, Docker, GHCR, Nginx blue/green, Prometheus/Grafana, and Slack/PagerDuty — with explicit justifications. The automated testing strategy uses a test pyramid with coverage and audit gates, the deployment strategy progresses code through DEV, STAGING, and PRODUCTION behind platform-enforced promotion gates, and security scans, quality checks, notifications, and three-level rollback mechanisms (image, traffic, and database) are incorporated directly into the workflow. Because every stage is automated and every release is reversible, the system can ship safely at the pace that a growing food-ordering business demands.

10. APPENDIX

A. SAMPLE CI/CD WORKFLOW CONFIGURATION (EXCERPT)

build-and-test	checkout → setup-node 20 → npm ci → npm run lint → npm test --coverage → upload artifact	ubuntu-latest
security-scan	trufflehog filesystem . → npm audit --audit-level=high → github/codeql- action/analyze	ubuntu-latest
docker-build- push	docker build -t ghcr.io/foodieconnect/app:\${{ sha }} → docker push	ubuntu-latest
deploy-dev	SSH to EC2 → docker pull + restart → smoke test /health	ubuntu-latest → EC2
deploy-staging	needs: deploy-dev; environment: staging (approval) → playwright run → report	ubuntu-latest → EC2
deploy- production	needs: deploy-staging; environment: production (lead approval) → blue/green via Nginx → 10-min watch → Slack notify	ubuntu-latest → EC2

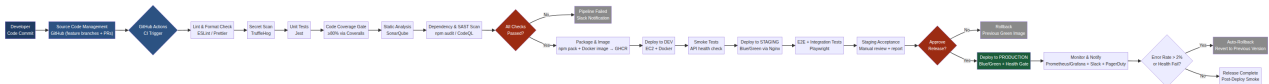
B. DEPLOYMENT ENVIRONMENT STATISTICS

CI workflow duration (average)	6 minutes (lint + scan + unit + build)
--------------------------------	--

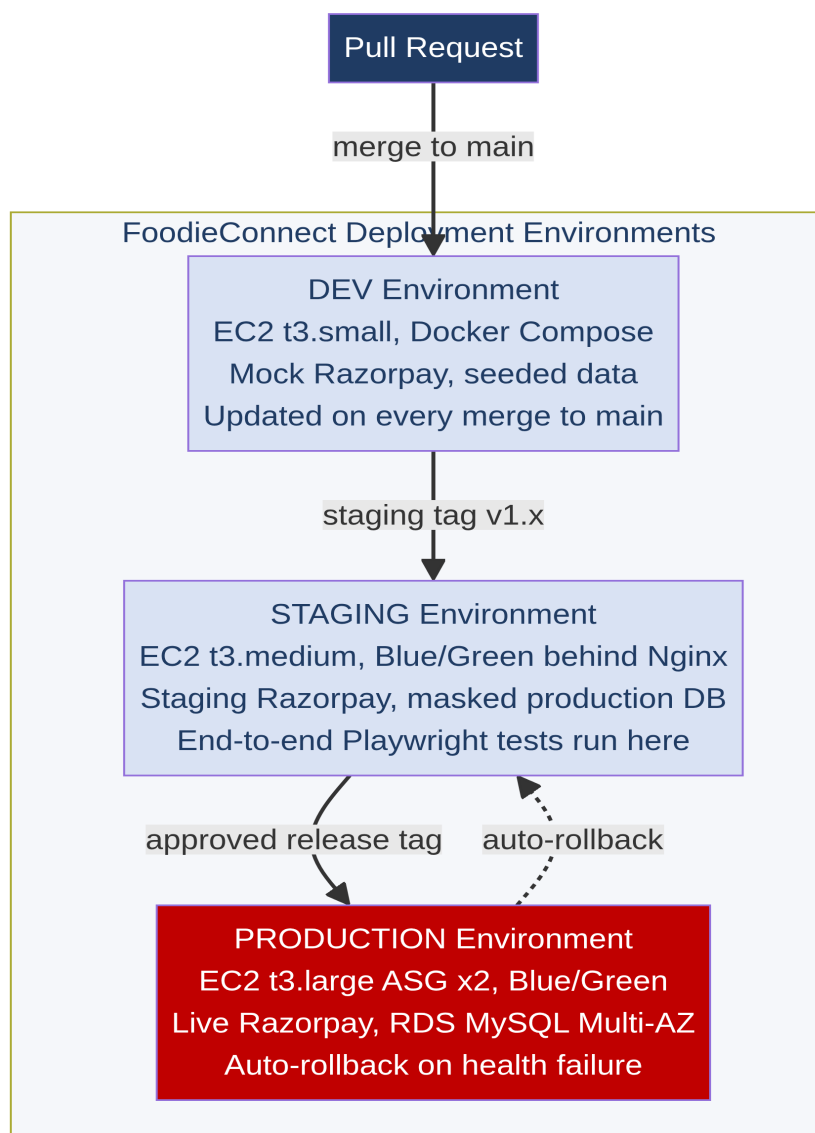
Staging promotion duration	12 minutes including E2E suite (23 tests)
Production cutover time	Under 1 minute (Nginx upstream switch)
Rollback time	Under 60 seconds (traffic-level revert)
Deployments per week (target)	4–6 through staging; 1–2 to production
Branches	main + feature branches (merged via PR)

C. KEY FIGURES

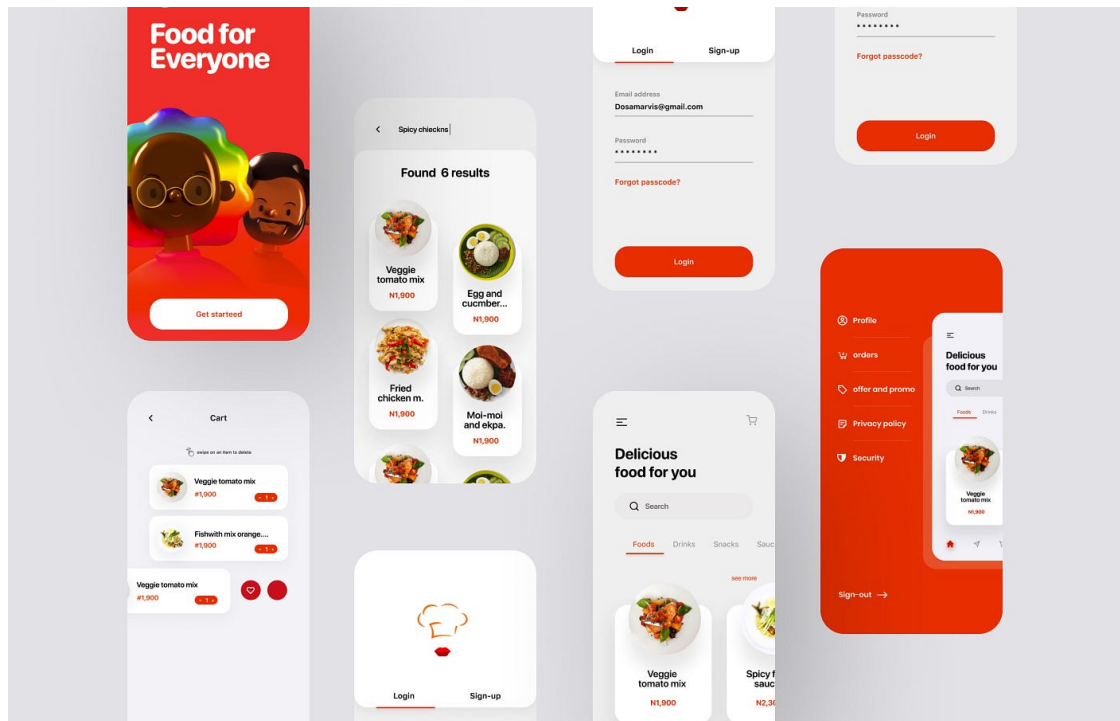
For completeness, the principal evidence figures are reproduced in the appendix.



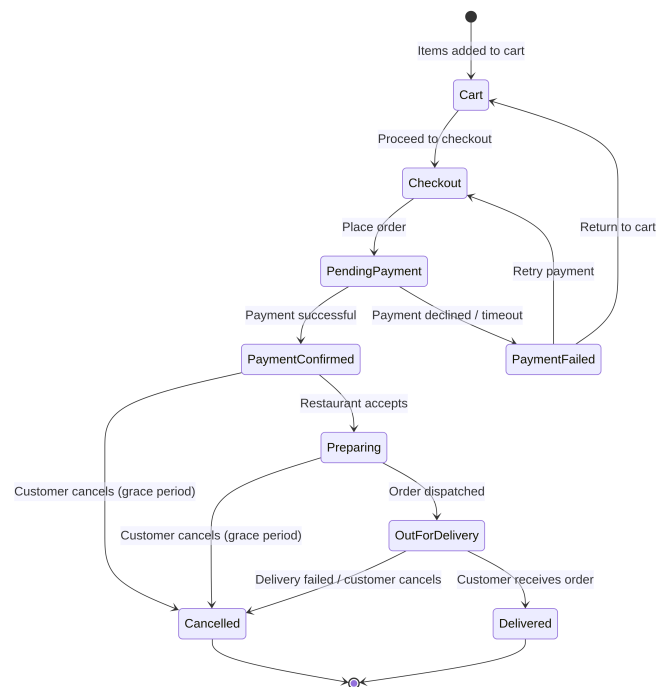
Appendix Figure C1: Full CI/CD pipeline diagram.



Appendix Figure C2: Deployment environment layout.



Appendix Figure C3: Representative screens of the system under delivery.



Appendix Figure C4: Order state model validated by the pipeline's unit and E2E tests